

CERTICHAIN

Blockchain-Based Academic Credential Issuance, Verification
and Fraud Detection Platform

PROJECT BLUEPRINT & TECHNICAL DOCUMENTATION

PROJECT TYPE	Final-Year Major Project
DOMAIN	Blockchain, Full-Stack Development, Mobile Application Development, Cybersecurity, Document Intelligence, OCR, Software Engineering
TARGET USERS	Universities, colleges, training institutes, students, recruiters/employers, verification authorities
PRIMARY PLATFORMS	Flutter Mobile App + Web Admin Portal + Public Verification Web Page
BACKEND	Java Spring Boot
INTELLIGENCE LAYER	Python
BLOCKCHAIN	Ethereum Sepolia Testnet
DATABASE	PostgreSQL
CLOUD	AWS
DEVELOPMENT DURATION	Approximately 4-5 months
TEAM	2 developers

Table of Contents

Part 1 — Project Overview	5
1. Executive Summary	5
2. Problem Statement	6
3. Main Objective	6
Part 2 — System Components & Architecture	7
4. What Exactly Are We Building?	7
5. Component 2 — Student Flutter Application	7
6. Component 3 — Public Verification Website	8
7. Component 4 — Python Document Intelligence Service	8
8. Complete System Architecture	9
Part 3 — Technology Stack	9
9. Frontend — Flutter	9
10. Web Frontend	9
11. Backend — Java Spring Boot	10
12. Database — PostgreSQL	10
13. Why PostgreSQL?	10
14. Blockchain Choice	11
15. Blockchain Development Stack	11
Part 4 — Blockchain Design & Core Workflows	12
16. What Exactly Goes on the Blockchain?	12
17. SHA-256 Hashing	12
18. Smart Contract Design	13
19. Certificate Issuance Workflow	14
20. Verification Workflow	14
21. Tampering Scenario	15
22. Certificate Revocation	15
Part 5 — Python Intelligence & Storage	16
23. Python Service	16
24. OCR Workflow	16
25. OCR Alternative for Flutter	17
26. Fraud Detection	17

27. Storage	18
Part 6 — APIs, Security & Data Model	19
28. API Sources	19
29. External APIs/Services	20
30. QR Code Design	20
31. Authentication	21
32. Database Structure	22
33. Certificate Lifecycle	23
34. REST API Design	23
35. Certificate Verification API Response	24
Part 7 — User Interfaces	24
36. Admin Dashboard	24
37. Student Application Screens	25
38. Public Verification Page	26
Part 8 — Security, Environment & Team	26
39. Security Architecture	26
40. Blockchain Security	27
41. Development Environment	27
42. Repository Structure	28
43. Git Workflow	28
44. Division of Work	29
45. Shared Responsibilities	29
Part 9 — Integration & Design Decisions	30
46. How the Services Communicate	30
47. Why Not Let Flutter Talk Directly to Blockchain?	31
48. Certificate Generation	31
49. Hashing Workflow	31
50. Why Have Both PostgreSQL and Blockchain?	32
51. Why Not Store Everything on Blockchain?	32
52. API Documentation	32
Part 10 — Testing & Deployment	33
53. Testing	33
54. Important Demonstration Test	33
55. Deployment Architecture	34

56. Docker	34
57. AWS Architecture	34
58. CI/CD	35

Part 11 — Scope, Features & Evaluation 35

59. External Services Summary	35
60. What We Are NOT Building	36
61. Minimum Viable Product	36
62. Advanced Features	37
63. Bulk Certificate Generation	37
64. Multi-Institution Architecture	38
65. Coursework Covered	39
66. Research Component	40
67. Evaluation Metrics	40
68. Demo Dataset	40

Part 12 — Timeline & Delivery 41

69. Project Timeline — Month 1: Foundation	41
70. Month 2: Certificate Engine	41
71. Month 3: Blockchain	41
72. Month 4: Python Intelligence	42
73. Month 5: Production & Research	42
74. Development Order	42
75. Final End-to-End Architecture	43
76. Final User Journey	43
77. What Each Person Can Put on Their Resume	44
78. Final Project Definition	45

PART 1

Project Overview

1. Executive Summary

CertiChain is a digital academic credential platform designed to allow educational institutions to issue, manage, store and verify tamper-evident digital certificates.

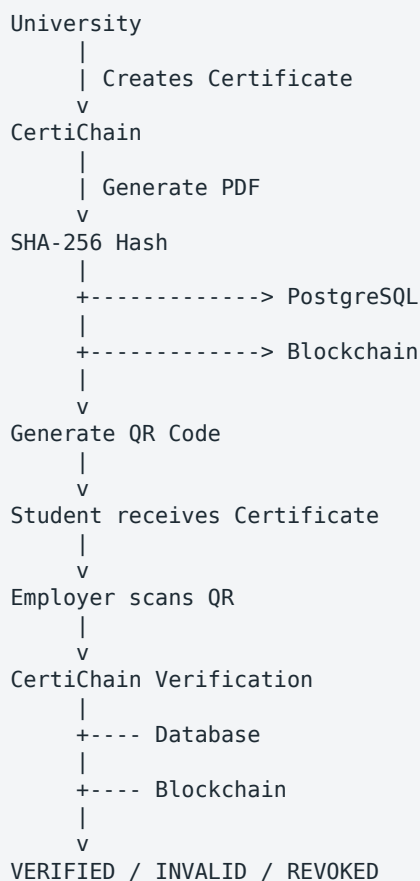
The core problem is that traditional digital certificates are often distributed as PDFs or images. A PDF can be modified after issuance, while manually verifying whether a certificate is genuine can require contacting the issuing institution.

CertiChain introduces a verification layer using cryptographic hashing and blockchain.

The actual certificate remains off-chain in secure storage. A cryptographic hash of the issued certificate is generated and recorded on the blockchain along with essential verification metadata.

When an employer or third party scans the certificate's QR code, CertiChain retrieves the certificate information and compares the uploaded/current document hash against the hash registered during issuance.

The basic workflow



The project additionally includes a Python document-intelligence service that can extract information from uploaded certificates using OCR and compare extracted information with the registered certificate record.

2. Problem Statement

Academic certificates are important proof of education, skills and qualifications. However, the verification process can have several problems:

- Certificates are frequently distributed as PDFs or printed documents.
- Digital documents can be edited.
- Recruiters may not have a direct verification mechanism.
- Manual verification can require communication with the issuing institution.
- Institutions may have different certificate formats and verification processes.
- QR codes alone do not prove authenticity because a QR code itself can be copied.
- Storing complete documents directly on a blockchain is inefficient and exposes unnecessary data.

CertiChain addresses these issues by combining

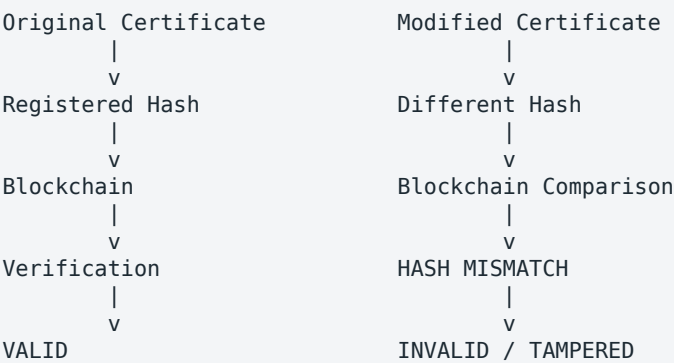
- Digital certificate generation
- SHA-256 cryptographic hashing
- Blockchain-based integrity records
- QR-based verification
- Role-based authentication
- Secure document storage
- OCR/document extraction
- Fraud/anomaly detection
- Centralized certificate management

3. Main Objective

The primary objective is to build a platform where an authorized institution can issue a digital certificate whose integrity can later be independently verified.

The system should answer: "Is this certificate the same certificate that was originally issued by the institution?"

The system should be able to distinguish between an original and a modified certificate:



PART 2

System Components & Architecture

4. What Exactly Are We Building?

CertiChain will consist of four major components.

Component 1 — University/Admin Web Portal

Used by authorized institutions. The administrator can:

- Register students
- Create student records
- Create certificates
- Upload certificate templates
- Generate certificates
- Issue certificates
- Revoke certificates
- View issued certificates
- Search certificates
- View verification history
- View blockchain transaction information
- Manage institution users

5. Component 2 — Student Flutter Application

The student receives access to their issued credentials. The mobile application can contain:

Main screens

- Dashboard
- My Certificates
- Verified Credentials
- Pending Credentials
- Revoked Credentials

Certificate details

- Certificate ID
- Student Name
- Institution
- Degree
- Department
- Issue Date
- Credential Type
- Verification Status
- Blockchain Transaction

Certificate actions

- View certificate
- Download certificate
- Share certificate
- Show QR code
- Scan verification QR
- View verification history

6. Component 3 — Public Verification Website

This is extremely important. The employer should not need to install the Flutter application. They can simply scan the QR code using their phone.

```
Certificate QR
  |
  v
https://certichain.example/verify/CERT-2026-001245
  |
  v
Public Verification Page
```

The page displays

```
CERTIFICATE VERIFICATION

Certificate ID:    CERT-2026-001245
Student:         Vanshika Nimwal
Institution:     Example University
Degree:         B.Tech Computer Science
Issue Date:      30 September 2026
Status:         VERIFIED
Blockchain Record: 0x8a91...42bc
Issued By:      Example University
```

No login should be required for basic public verification.

7. Component 4 — Python Document Intelligence Service

This is where your brother's Python expertise becomes important. The Python service handles OCR.

If someone uploads a certificate image/PDF:

```
Certificate
  |
  v
Python OCR
  |
  v
Extract Text
```

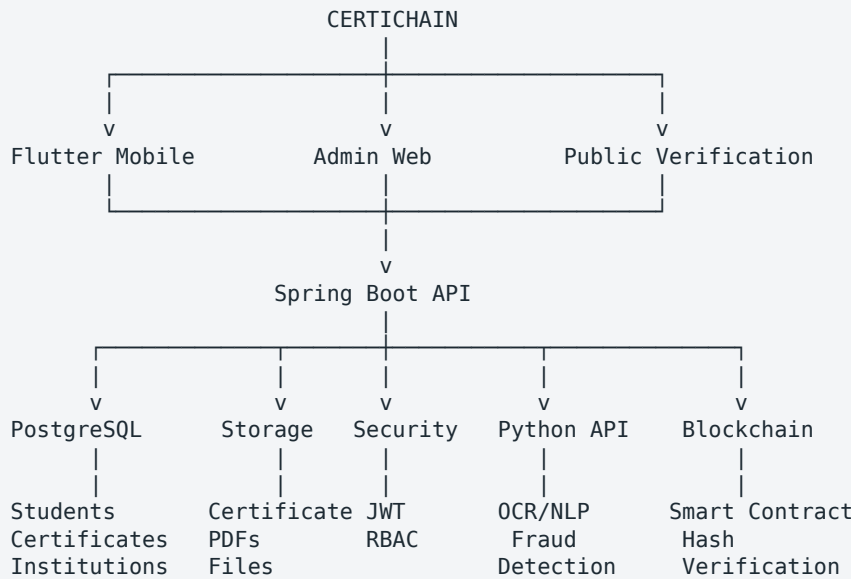
It can extract

- Student name
- Certificate ID
- Institution
- Degree
- Date
- Registration number
- Grade/CGPA if present

Then it compares the extracted values with the official certificate record.

8. Complete System Architecture

The proposed architecture is:



PART 3

Technology Stack

9. Frontend — Flutter

Flutter is used for:

- Student application
- Certificate wallet
- QR scanner
- Certificate viewing
- Authentication
- Profile
- Notifications

Suggested Flutter packages

- | | | |
|--------------------|--------------------------|------------------------|
| • dio | • flutter_secure_storage | • file_picker |
| • flutter_riverpod | • mobile_scanner | • cached_network_image |
| • go_router | • qr_flutter | • firebase_messaging |

Firebase Cloud Messaging can provide push notifications for Android, iOS and web clients.

10. Web Frontend

You have two possible choices.

Option A — React (recommended for the admin portal)

- | | |
|--------------|----------------|
| • React | • Tailwind CSS |
| • TypeScript | • Axios |
| • Vite | • React Router |

Option B — Flutter Web

If you want to reduce the number of technologies: Flutter Mobile + Flutter Web. Since you already know Flutter, this is easier.

However, for a stronger software-engineering resume, React + TypeScript + Flutter demonstrates broader frontend experience.

11. Backend — Java Spring Boot

This will be the primary backend. Spring Boot is designed for production-grade Java applications and provides support around web applications, security, metrics, health checks and configuration.

Use

- Java
- Spring Boot
- Spring Web
- Spring Data JPA
- Hibernate
- Spring Security
- JWT
- Bean Validation
- PostgreSQL Driver
- Lombok
- OpenAPI/Swagger

Spring Data JPA provides the persistence layer for mapping Java entities to relational database tables. Spring Security will handle authentication and authorization.

12. Database — PostgreSQL

We will use PostgreSQL as the main relational database. Example tables:

```
users
institutions
students
certificates
certificate_templates
certificate_versions
verification_logs
blockchain_records
revocations
audit_logs
```

13. Why PostgreSQL?

The data is highly relational. A relational database is therefore appropriate.

```
Institution
|
+---- Users
|
+---- Students
|
+---- Certificates
      |
      +---- Verification Records
      |
      +---- Blockchain Record
```

14. Blockchain Choice

For the academic prototype, we recommend using Ethereum Sepolia Testnet. Sepolia is currently an Ethereum public testnet intended for smart-contract and application development. Its chain ID is 11155111.

We should not use Ethereum mainnet for the project because transactions would involve real assets/costs.

```
Ethereum Sepolia
  |
  v
Alchemy RPC
  |
  v
Smart Contract
```

Alchemy provides Ethereum Sepolia RPC endpoints such as:

```
https://eth-sepolia.g.alchemy.com/v2/<API_KEY>
```

and identifies Sepolia with chain ID 11155111.

15. Blockchain Development Stack

Your brother can use:

- Solidity
- Hardhat
- Ethers.js or viem
- OpenZeppelin Contracts
- Alchemy
- Ethereum Sepolia

OpenZeppelin provides reusable Solidity contracts and access-control components. The smart-contract development flow can be:

```
Solidity -> Hardhat -> Compile -> Test -> Deploy -> Sepolia -> Contract Address
```

OpenZeppelin also provides tooling and documentation for developing, testing and deploying smart contracts.

PART 4

Blockchain Design & Core Workflows

16. What Exactly Goes on the Blockchain?

This is extremely important. We should NOT store the PDF on the blockchain. That would be inefficient and could expose personal information.

Instead, store a minimal verification record:

- certificateId
- certificateHash
- issuerId / issuerReference
- issueTimestamp
- status

Conceptually:

CERT-2026-001245

Hash: A3F92E7D...7BC21
Issuer: University-ID-001
Timestamp: 2026-09-30
Status: ACTIVE

The actual certificate remains in secure off-chain storage.

17. SHA-256 Hashing

Suppose the generated certificate is certificate.pdf. The backend calculates SHA-256(certificate.pdf), for example A3F92E7D1C...7BC21. That becomes the certificate's integrity fingerprint.

If even a small portion of the PDF changes:

Version	Content	Hash
Original	CGPA = 8.2	A3F92E...
Modified	CGPA = 9.2	91BD4C...

The hashes no longer match. Therefore the system asks: is Blockchain Hash = Current Certificate Hash? If yes, integrity is verified. If no, an integrity mismatch is reported.

18. Smart Contract Design

We don't need an NFT. We don't need cryptocurrency. We don't need ERC-20 tokens. We need a simple Certificate Registry Smart Contract.

Conceptual functions

```
registerCertificate(  
    certificateId,  
    certificateHash,  
    issuerId  
)  
  
verifyCertificate(  
    certificateId,  
    certificateHash  
)  
  
revokeCertificate(  
    certificateId  
)
```

The contract can maintain

```
Certificate ID  
  |  
  v  
Certificate Record  
  |  
  ├── Hash  
  ├── Issuer  
  ├── Timestamp  
  └── Status
```

OpenZeppelin's access-control functionality can be used to restrict administrative blockchain operations.

19. Certificate Issuance Workflow

This is the most important workflow in the project.

Step	Action
1	University admin logs in: POST /api/auth/login
2	Admin selects Create Certificate
3	Admin enters Student Name, Student ID, Degree, Department, Certificate Type, Issue Date, CGPA/Grade
4	Spring Boot validates the information
5	Certificate PDF is generated, e.g. CERT-2026-001245.pdf
6	PDF is uploaded to secure storage
7	Backend calculates SHA-256
8	Backend sends the hash to the blockchain service
9	Smart contract records Certificate ID, Hash, Issuer, Timestamp
10	Blockchain returns the Transaction Hash
11	Backend stores certificate_id, hash, transaction_hash, contract_address, block_number
12	QR code is generated; it contains a verification URL: https://certichain.example/verify/CERT-2026-001245
13	Student receives the certificate

20. Verification Workflow

An employer scans the QR code.

```
QR
|
v
Verification URL
|
v
Spring Boot
|
v
Certificate ID
|
+-----> PostgreSQL
|
+-----> Blockchain
|
v
Compare Records
|
v
Verification Result
```

Example

```
Certificate ID:    CERT-2026-001245
Database Hash:    A3F92...
Blockchain Hash:  A3F92...
Status:           ACTIVE

Result:           VERIFIED
```

21. Tampering Scenario

Suppose somebody downloads CERT-2026-001245.pdf and changes CGPA: 8.2 to CGPA: 9.2. The PDF now has a different hash.

Hash	Value
Original Hash	A3F92...
Modified Hash	91BD4...

The system detects Blockchain Hash != Submitted Certificate Hash and displays:

```
VERIFICATION FAILED
```

```
Reason:
Certificate integrity mismatch.
```

```
The submitted document does not match
the certificate registered at issuance.
```

22. Certificate Revocation

This is another important feature. Suppose an institution accidentally issues a certificate or discovers an administrative problem. The university can revoke it: ACTIVE becomes REVOKED. The public verification page should then show Certificate Status: REVOKED.

This is different from tampering.

Three states

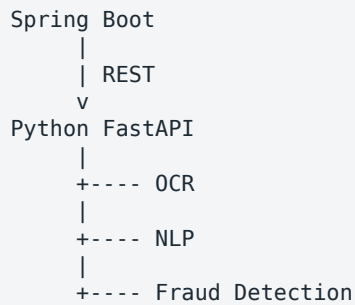
State	Meaning
ACTIVE	Certificate is registered and valid
REVOKED	Certificate was withdrawn by the institution
INVALID / HASH MISMATCH	Submitted document does not match the registered hash

PART 5

Python Intelligence & Storage

23. Python Service

The Python service should be a separate microservice.



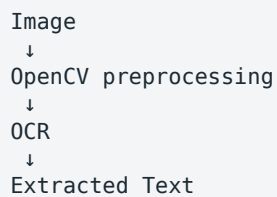
Use

- Python
- FastAPI
- OpenCV
- PyMuPDF
- pytesseract
- Pandas
- scikit-learn

Potentially use a document OCR service where appropriate, but for the academic prototype, local/open-source OCR keeps the project reproducible.

24. OCR Workflow

User uploads certificate.jpg. Python receives it.



Example extracted text

Name: Vanshika Nimwal
University: Example University
Degree: B.Tech CSE
Certificate ID: CERT-2026-001245
Issue Date: 30/09/2026

Python returns structured JSON

```
{  
  "name": "Vanshika Nimwal",  
  "university": "Example University",  
  "degree": "B.Tech CSE",  
  "certificateId": "CERT-2026-001245",  
  "issueDate": "2026-09-30"  
}
```

Spring Boot then compares those values with the official record.

25. OCR Alternative for Flutter

If we want fast mobile-side scanning, Google ML Kit can also be considered. Google's ML Kit provides on-device text recognition and barcode scanning capabilities, among other vision APIs.

For the final architecture, however, we recommend:

```
QR scanning    -> Flutter  
Document OCR   -> Python
```

This keeps responsibilities clean.

26. Fraud Detection

Do not claim that the system can magically determine whether a certificate is fraudulent. Instead, build a document consistency/anomaly detection module.

```
Uploaded Certificate  
  |  
  v  
OCR  
  |  
  v  
Extract Fields  
  |  
  v  
Compare With Database  
  |  
  |— Certificate ID exists?  
  |— Name matches?  
  |— Institution matches?  
  |— Degree matches?  
  |— Date matches?  
  |— Status active?  
  |  
  v  
Verification Report
```

Example report

Field	Result
Certificate ID	MATCH
Student Name	MATCH
Institution	MATCH
Degree	MATCH
Issue Date	MISMATCH
Blockchain Hash	MISMATCH

Result: SUSPICIOUS. This is much more academically defensible than claiming "AI detects fake certificates."

27. Storage

We have two choices.

Option A — AWS S3 (recommended for the production-style architecture)

AWS S3 is object storage designed for storing files such as documents and application assets.



S3 supports private buckets and controlled access through IAM and related mechanisms.

Option B — Cloudinary

Simpler for student development. Cloudinary supports programmatic file uploads and PDF management through APIs.

For the final architecture, AWS S3 is recommended because it gives additional cloud/backend experience.

PART 6

APIs, Security & Data Model

28. API Sources

The question: where are we getting the APIs from? The answer: most of our important APIs are our own APIs. We aren't going to find a random "certificate API."

Spring Boot exposes

```
/api/auth  
/api/students  
/api/institutions  
/api/certificates  
/api/verification  
/api/admin  
/api/revocations  
/api/blockchain  
/api/audit
```

Python exposes

```
/api/ocr  
/api/document/analyze  
/api/fraud/check
```

Blockchain interaction uses Ethereum JSON-RPC through Alchemy. Ethereum applications communicate with blockchain nodes through JSON-RPC, and Alchemy provides Ethereum Sepolia RPC infrastructure.

29. External APIs/Services

Requirement	Technology/API
Blockchain	Ethereum Sepolia
Blockchain RPC	Alchemy
Smart contract	Solidity
Contract libraries	OpenZeppelin
Database	PostgreSQL
File storage	AWS S3
OCR	Python/Tesseract
Image processing	OpenCV
QR generation	Flutter package / backend library
QR scanning	Mobile Scanner / ML Kit
Push notifications	Firebase Cloud Messaging
Authentication	Spring Security + JWT
API documentation	OpenAPI/Swagger
Maps	Optional; Google Maps
Email	Optional transactional email provider
Hosting	AWS
Containerization	Docker
CI/CD	GitHub Actions

Springdoc can automatically generate OpenAPI/Swagger documentation from a Spring Boot application.

30. QR Code Design

The QR code should not contain the entire certificate. It should contain something like:

```
https://certichain.example/verify/CERT-2026-001245
```

When scanned:

```
QR -> Verification URL -> Certificate ID -> Backend -> Database + Blockchain
```

This prevents sensitive information from being embedded directly in the QR.

31. Authentication

Use JWT authentication.

Roles

- SUPER_ADMIN
- INSTITUTION_ADMIN
- ISSUER
- STUDENT
- VERIFIER

Role	Capabilities
Admin	Create certificate, revoke certificate, view certificates
Student	View own certificates, download, share
Public verifier	Verify certificate

Spring Security supports web security and authorization features including method-level security.

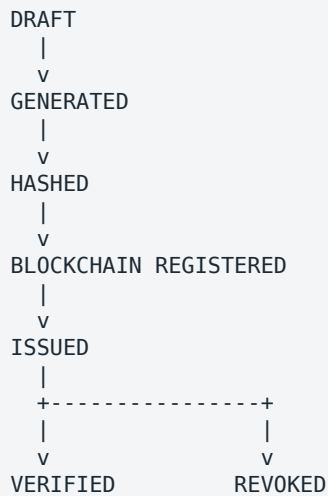
32. Database Structure

A simplified schema:

users id name email password_hash role institution_id created_at	institutions id name registration_number email created_at
students id user_id student_number department course graduation_year	certificates id certificate_id student_id institution_id certificate_type degree issue_date document_url document_hash status created_at
blockchain_records id certificate_id contract_address transaction_hash block_number blockchain_hash network registered_at	revocations id certificate_id reason revoked_by revoked_at
verification_logs id certificate_id verification_type result ip_hash verified_at	audit_logs id user_id action entity entity_id timestamp

33. Certificate Lifecycle

The complete lifecycle:



34. REST API Design

Some important endpoints:

Authentication

```
POST /api/auth/register
POST /api/auth/login
POST /api/auth/refresh
POST /api/auth/logout
```

Students

```
GET /api/students/{id}
POST /api/students
PUT /api/students/{id}
GET /api/students/{id}/certificates
```

Certificates

```
POST /api/certificates
GET /api/certificates/{id}
GET /api/certificates/student/{studentId}
POST /api/certificates/{id}/issue
POST /api/certificates/{id}/revoke
```

Verification

```
GET /api/verify/{certificateId}
POST /api/verify/document
```

Blockchain

```
POST /api/blockchain/register
GET /api/blockchain/{certificateId}
```

Python

```
POST /api/ocr/extract
POST /api/document/analyze
POST /api/fraud/check
```

35. Certificate Verification API Response

Example:

```
{
  "certificateId": "CERT-2026-001245",
  "status": "VERIFIED",
  "student": "Vanshika Nimwal",
  "institution": "Example University",
  "degree": "B.Tech CSE",
  "issueDate": "2026-09-30",
  "hashMatch": true,
  "blockchainVerified": true,
  "revoked": false,
  "transactionHash": "0x8a91..."
}
```

PART 7

User Interfaces

36. Admin Dashboard

The admin dashboard should show:

CERTICHAIN ADMIN

Total Students	4,832
Certificates Issued	12,482
Verified Certificates	11,903
Revoked Certificates	32
Pending Certificates	81

Charts

- Certificates by month
- Certificates by department
- Verification attempts
- Failed verification attempts
- Revocations
- Certificate types

37. Student Application Screens

The Flutter application can contain the following screens.

Authentication

- Login
- Register
- Forgot Password

Dashboard

Hello, Vanshika

Certificates	3
Verified	3
Revoked	0

My Certificates

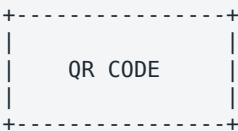
B.Tech Degree Certificate
30 Sep 2026
VERIFIED

Certificate Detail

Certificate, Student, Institution, Degree, Issue Date, Certificate ID, with actions:

[View Certificate] [Share] [Show QR]

QR screen



Scan to verify

38. Public Verification Page

This should be extremely simple.

CERTICHAIN

Certificate Verification

Certificate ID:	CERT-2026-001245
Student:	Vanshika Nimwal
Institution:	Example University
Degree:	B.Tech CSE
Issue Date:	30 September 2026
Status:	VERIFIED
Blockchain:	Verified
Document Integrity:	Verified
Transaction:	0x8a91...

[\[View Certificate\]](#)

PART 8

Security, Environment & Team

39. Security Architecture

Important security practices:

Passwords

Never store plaintext passwords. Use BCrypt / Argon2.

JWT

Use an Access Token and a Refresh Token.

Documents

Keep the S3 bucket private.

Blockchain

Never store the following on-chain unless there is a carefully justified reason:

- Student address
- Phone number
- Email
- Full certificate PDF
- Personal documents

Store only the minimum integrity/verification metadata.

API

Implement:

- Authentication
- Authorization
- Input validation
- Rate limiting
- CORS
- HTTPS
- Audit logging

40. Blockchain Security

The smart contract should have authorization. Only an authorized issuer should be able to call `registerCertificate()` and `revokeCertificate()`. A public user should only be able to call `verifyCertificate()`.

OpenZeppelin provides reusable access-control components that can be used for this type of authorization.

41. Development Environment

Your computer

- Java JDK
- IntelliJ IDEA
- Flutter
- Android Studio
- Node.js
- Git
- Docker
- PostgreSQL
- Postman
- VS Code

Brother's computer

- Python
- PyCharm / VS Code
- Node.js
- Hardhat
- Git
- Docker

Both

- Git
- GitHub
- Postman
- Docker

42. Repository Structure

We recommend a monorepo:

```
certichain/
├── mobile/
│   └── Flutter application
├── admin-web/
│   └── React/Flutter web
├── backend/
│   └── Spring Boot
├── python-service/
│   └── FastAPI + OCR + ML
├── blockchain/
│   ├── contracts/
│   ├── scripts/
│   ├── test/
│   └── hardhat.config.js
├── docs/
│   ├── architecture/
│   ├── api/
│   ├── database/
│   └── research/
├── deployment/
│   ├── docker/
│   └── nginx/
└── README.md
```

43. Git Workflow

Use the branches `main`, `develop` and `feature/*`. Example feature branches:

```
feature/authentication
feature/certificate-generation
feature/qr-verification
feature/blockchain-contract
feature/ocr-service
feature/flutter-wallet
```

This also gives you a proper software-engineering story during interviews.

44. Division of Work

This needs to be very clear because you are two developers.

Your responsibilities

Area	Scope
Frontend	Flutter, Admin UI, Student UI, QR scanner, Certificate wallet, Verification UI
Backend	Spring Boot, REST APIs, Authentication, JWT, PostgreSQL, Certificate management, File management, Verification logic, Audit logs
DevOps	Docker, AWS, CI/CD, Deployment, Monitoring

Brother's responsibilities

Area	Scope
Blockchain	Solidity, Smart Contract, Hardhat, Ethereum Sepolia, Alchemy, Contract testing, Blockchain integration
Python	FastAPI, OCR, OpenCV, Document processing, Text extraction, Fraud/anomaly detection

45. Shared Responsibilities

Both should participate in:

- System architecture
- Database design
- Security
- Testing
- Documentation
- Research paper
- Final presentation
- Deployment
- GitHub

This is important because both members should understand the complete system before the viva.

PART 9

Integration & Design Decisions

46. How the Services Communicate

The system uses REST APIs.

```
Flutter
  |
  | HTTPS
  |
  v
Spring Boot
  |
  +----- PostgreSQL
  |
  +----- AWS S3
  |
  +----- Python
  |
  +----- Blockchain
```

Python

```
Spring Boot
  |
  | HTTP POST
  |
  v
FastAPI
  |
  | OCR / Analysis
  |
  | JSON
  |
  v
Spring Boot
```

Blockchain

```
Spring Boot
  |
  | Blockchain service
  |
  | Alchemy RPC
  |
  | Ethereum Sepolia
  |
  | Smart Contract
```

47. Why Not Let Flutter Talk Directly to Blockchain?

Because it would complicate security and architecture. Instead:

```
Flutter -> Spring Boot -> Blockchain
```

The backend controls:

- authorization
- transaction preparation
- issuer identity
- certificate validation
- database synchronization

The mobile app remains a client.

48. Certificate Generation

The backend can generate certificates from an HTML/template.

```
Certificate Data
  |
  v
HTML Template
  |
  v
PDF
```

Java libraries/options can be evaluated for PDF generation.

The final PDF becomes the exact artifact that gets hashed. The hash should be generated after the final certificate PDF has been created.

49. Hashing Workflow

```
Certificate Data
  |
  v
Generate PDF
  |
  v
Final PDF
  |
  v
SHA-256
  |
  v
Document Hash
  |
  +-----+
  |       |
  v       v
PostgreSQL Blockchain
```

Therefore: Database Hash = Blockchain Hash.

50. Why Have Both PostgreSQL and Blockchain?

This is a likely viva question.

Layer	Stores
PostgreSQL	Application data: Student, Certificate, Institution, Document URL, User, Verification logs
Blockchain	Tamper-evident verification data: Certificate ID, Certificate Hash, Issuer, Timestamp, Status

Blockchain is not a replacement for the database. It is an integrity layer.

51. Why Not Store Everything on Blockchain?

Because certificates may contain personal information. Also, blockchain data is not intended to be used like a normal document database. Therefore:

```
Large/private data -> PostgreSQL / S3
Integrity proof    -> Blockchain
```

This is one of the most important architectural decisions in the project.

52. API Documentation

Use Swagger/OpenAPI. The backend will expose `/swagger-ui`. Developers can see and test these endpoints directly:

```
POST /api/auth/login
POST /api/certificates
GET  /api/certificates/{id}
GET  /api/verify/{id}
```

Springdoc OpenAPI can generate OpenAPI documentation and Swagger UI from Spring Boot applications.

PART 10

Testing & Deployment

53. Testing

The project should have several testing levels.

Backend — JUnit, Mockito, Spring Boot Test

- Authentication
- Certificate creation
- Certificate issuing
- Certificate revocation
- Verification
- Hash calculation
- Authorization

Flutter

- Unit tests
- Widget tests
- Integration tests

Python — pytest

- OCR
- Text extraction
- Field matching
- Document analysis

Blockchain — Hardhat tests

- Register certificate
- Duplicate certificate
- Unauthorized issuer
- Verify certificate
- Revoke certificate

54. Important Demonstration Test

During your final presentation, perform this live.

Test 1 — Genuine certificate

Certificate PDF -> Scan QR -> Verification -> VERIFIED

Test 2 — Tampered certificate

Open the PDF. Change 8.2 to 9.2. Save it. Upload it for verification.

Modified PDF -> SHA-256 -> Different Hash -> Blockchain comparison -> FAILED

Test 3 — Revoked certificate

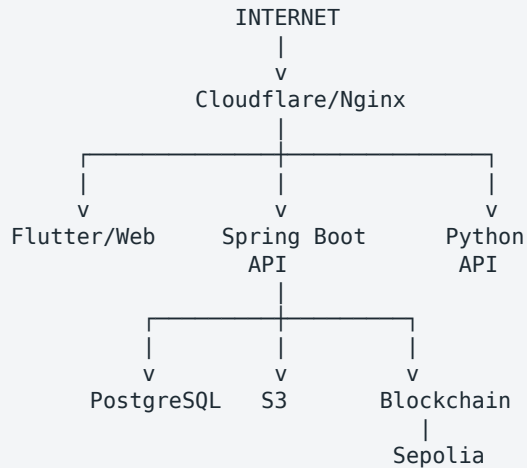
Admin revokes the certificate. Scan the QR again.

Certificate -> REVOKED

This three-part demonstration will make the purpose of the project immediately understandable.

55. Deployment Architecture

For the final deployed version:



56. Docker

Create containers for: backend, python-service, frontend, postgres, nginx.

Development

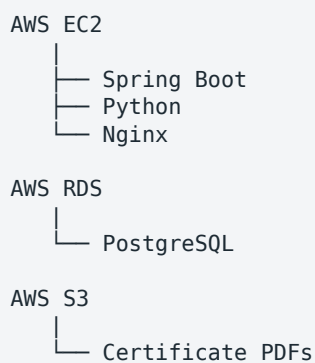
`docker compose up`

Production

`Docker -> AWS`

57. AWS Architecture

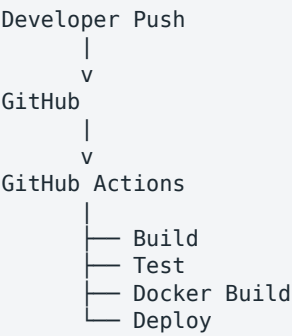
A realistic student deployment could be:



Later, CloudFront, Route 53 and CloudWatch can be added. S3 is specifically designed for object storage and supports application access through APIs and SDKs.

58. CI/CD

Use GitHub Actions.



For example, on main:

main -> CI -> JUnit -> Python Tests -> Flutter Build -> Docker -> AWS

PART 11

Scope, Features & Evaluation

59. External Services Summary

Purpose	Service
Blockchain	Alchemy Ethereum Sepolia RPC (Sepolia is Ethereum's public testnet for smart-contract development, chain ID 11155111)
Smart Contract	OpenZeppelin Contracts
Cloud Storage	Amazon S3
Notifications	Firebase Cloud Messaging
OCR / on-device scanning option	Google ML Kit
API Documentation	Springdoc OpenAPI

60. What We Are NOT Building

This needs to be explicitly defined so the project doesn't become too large. We are not trying to build:

- A replacement for DigiLocker
- A government certificate repository
- A real national academic database
- A cryptocurrency
- An NFT marketplace
- A blockchain wallet
- A complete university ERP
- A legally authoritative certificate authority
- A system that can automatically prove every document is genuine

We are building a technical prototype for issuing, managing and independently verifying academic credentials using cryptographic hashes, blockchain-backed integrity records, QR-based verification and document intelligence.

61. Minimum Viable Product

If time becomes a problem, these are the mandatory components:

#	Component
1	Admin Login
2	Student Registration
3	Certificate Creation
4	PDF Generation
5	SHA-256 Hashing
6	Blockchain Registration
7	QR Generation
8	Public Verification
9	Certificate Revocation
10	Flutter Student App
11	Spring Boot API
12	PostgreSQL

That alone is a complete final-year project.

62. Advanced Features

If the core system is completed early:

Level	Feature
Level 1	OCR
Level 2	Document consistency analysis
Level 3	Fraud/anomaly detection
Level 4	Analytics dashboard
Level 5	Multi-institution support
Level 6	Certificate templates
Level 7	Bulk certificate issuance
Level 8	Email/push notifications
Level 9	Certificate sharing links
Level 10	Employer verification portal

63. Bulk Certificate Generation

This would be an excellent feature for demonstrating real-world usefulness. The university uploads students.csv, for example:

```
student_id,name,degree,department,cgpa
001,Vanshika Nimwal,B.Tech CSE,CSE,8.2
002,Student B,B.Tech CSE,CSE,8.7
003,Student C,B.Tech CSE,CSE,9.1
```

The system then runs:

```
CSV
↓
Validate
↓
Generate certificates
↓
Hash each
↓
Register each
↓
Generate QR
↓
Store
```

Now you are demonstrating batch processing as well.

64. Multi-Institution Architecture

Eventually:

CERTICHAIN

- University A
- University B
- College C
- Training Institute D

Each institution gets: Institution Admin, Institution ID, Authorized Issuers, Certificate Templates and Certificate Records. The same backend supports multiple organizations.

This introduces multi-tenancy, RBAC and data isolation, which is excellent backend coursework.

65. Coursework Covered

This project can map directly to a large portion of a CSE curriculum.

Coursework	Where Used
Data Structures	Backend collections, algorithms
DBMS	PostgreSQL
OOP	Java/Spring architecture
Operating Systems	Deployment/processes
Computer Networks	HTTP/HTTPS/REST
Web Development	Admin/Public portal
Mobile Development	Flutter
Software Engineering	Architecture, SDLC
Software Testing	JUnit/PyTest/Flutter tests
Cybersecurity	Authentication, hashing, authorization
Cryptography	SHA-256, digital integrity
Cloud Computing	AWS
Distributed Systems	Backend + Python + blockchain
AI/ML	OCR/anomaly detection
Computer Vision	Document processing
Blockchain	Smart contract
DevOps	Docker + CI/CD
API Development	REST/OpenAPI
Database Management	PostgreSQL
UI/UX	Flutter/Web
Data Analytics	Admin dashboard
Computer Architecture	Indirectly through deployment/runtime
Information Security	Document integrity/access control

66. Research Component

The project can also produce a research paper.

Possible title

CertiChain: A Blockchain-Backed Framework for Tamper-Evident Academic Credential Verification Using Cryptographic Hashing and Document Intelligence

Possible research questions

- Can blockchain-backed hashes provide an independent integrity verification layer for digital academic certificates?
- What information should remain off-chain?
- How effectively can OCR extract structured information from certificate documents?
- How can document-field mismatches be identified?
- What are the performance costs of blockchain-based certificate registration?
- How does verification latency compare between database-only and blockchain-backed verification?

67. Evaluation Metrics

You should collect actual experimental results rather than inventing them. Measure:

Area	Metrics
Backend	API response time, throughput, error rate
Blockchain	Transaction confirmation time, gas consumption, registration latency, verification latency
OCR	Text extraction accuracy, field extraction accuracy, processing time
Verification	Valid certificate detection, modified certificate detection, revoked certificate detection
System	Concurrent users, database query latency, document upload time

68. Demo Dataset

Since you won't have access to real university records initially, create a clearly labelled synthetic dataset, for example: Demo University, Demo Institute, Demo Student, Demo Certificate.

Do not represent fabricated certificates as genuine real-world credentials.

For the final demo, you can say: "The prototype uses synthetic academic records because integration with an actual university issuing authority is outside the scope of the academic project."

PART 12

Timeline & Delivery

69. Project Timeline — Month 1: Foundation

Week	Focus
Week 1	Requirement analysis, system architecture, database design, GitHub setup
Week 2	Spring Boot setup, PostgreSQL, Authentication, JWT
Week 3	Flutter project, Admin portal, Student module
Week 4	Certificate data model, Certificate CRUD, API documentation

70. Month 2: Certificate Engine

Week	Focus
Week 5	Certificate templates, PDF generation
Week 6	SHA-256 hashing, Secure storage
Week 7	QR generation, Public verification page
Week 8	End-to-end certificate issuance

At this point the following should work without blockchain:

```
Admin -> Certificate -> Hash -> QR -> Verification
```

71. Month 3: Blockchain

Week	Focus
Week 9	Brother: Solidity fundamentals, Hardhat, Local blockchain
Week 10	CertificateRegistry contract
Week 11	Contract testing
Week 12	Sepolia deployment, Alchemy integration

Then Spring Boot to Blockchain works.

72. Month 4: Python Intelligence

Week	Focus
Week 13	FastAPI, OCR, PDF processing
Week 14	Text extraction, Field extraction
Week 15	Document comparison, Anomaly detection
Week 16	Spring Boot ↔ Python integration

73. Month 5: Production & Research

Week	Focus
Week 17	Security
Week 18	Testing
Week 19	Docker, AWS, CI/CD
Week 20	Performance testing, Documentation, Research paper, Final presentation

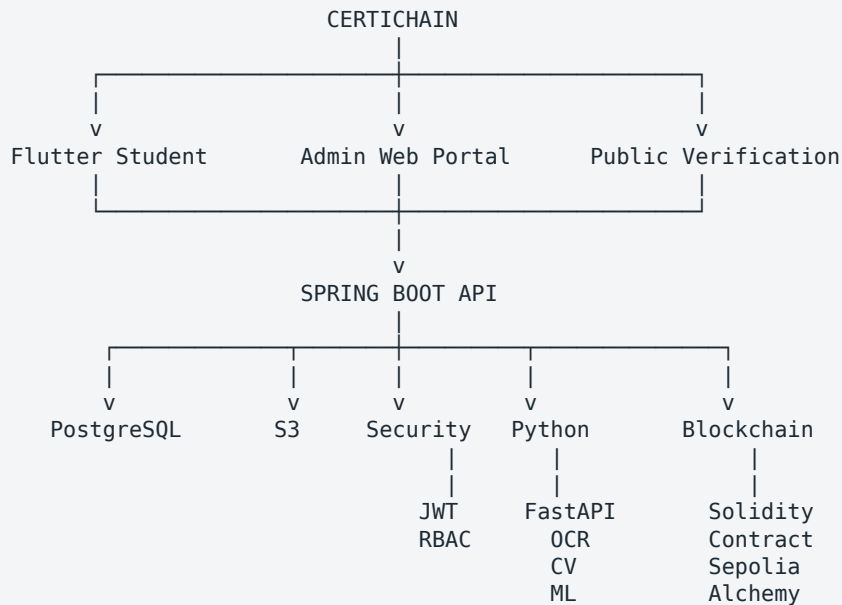
74. Development Order

This is important. Do not start with blockchain. Build in this order, which prevents the project from collapsing because of blockchain integration problems.

Order	Stage
1	Database
2	Spring Boot APIs
3	Authentication
4	Certificate generation
5	Hashing
6	QR verification
7	Flutter application
8	Blockchain
9	Python OCR
10	Fraud analysis
11	Docker
12	AWS
13	Testing

75. Final End-to-End Architecture

The final system should look approximately like this:



76. Final User Journey

University

Login -> Select Student -> Create Certificate -> Generate PDF -> SHA-256 -> Blockchain -> QR -> Issue

Student

Login -> My Certificates -> View Certificate -> Share QR

Employer

Scan QR
 ↓
 Open Verification Website
 ↓
 Certificate ID
 ↓
 Database Lookup
 ↓
 Blockchain Lookup
 ↓
 Hash Verification
 ↓
 Result

Suspicious Document

```
Upload Document
↓
Python OCR
↓
Extract Information
↓
Compare Official Record
↓
Hash Comparison
↓
Mismatch
↓
SUSPICIOUS / INVALID
```

77. What Each Person Can Put on Their Resume

Your resume

CertiChain — Blockchain-Based Academic Credential Verification Platform. Developed a full-stack academic credential platform using Flutter, Java Spring Boot, PostgreSQL, AWS and REST APIs, implementing role-based authentication, digital certificate generation, QR-based verification, document management and blockchain-backed integrity validation.

Your brother's resume

CertiChain — Blockchain-Based Academic Credential Verification Platform. Developed Solidity smart contracts and a Python document-intelligence service for blockchain-backed certificate integrity verification, OCR-based information extraction and document consistency analysis using Ethereum Sepolia, FastAPI and computer-vision/NLP techniques.

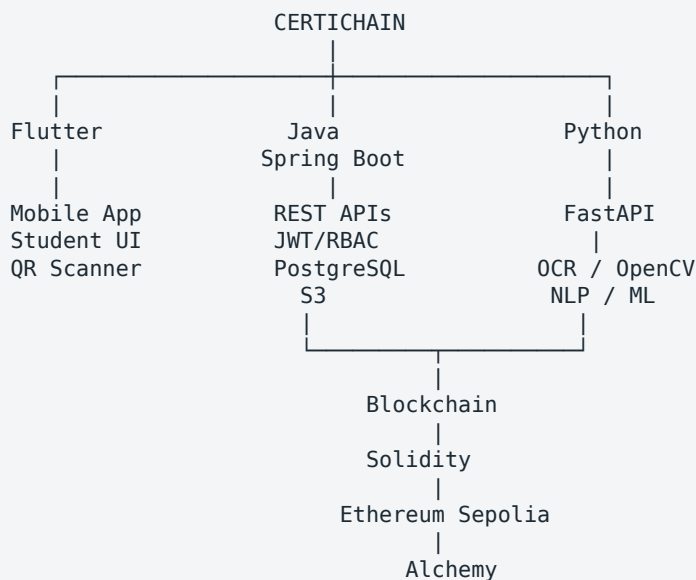
These are different contributions to the same project, which is exactly what you want as a two-person team.

78. Final Project Definition

If you need to explain the project in one paragraph to your professor:

CertiChain is a full-stack academic credential management and verification platform that enables authorized educational institutions to digitally issue certificates and allows students and third-party verifiers to independently validate their authenticity. The system generates a cryptographic SHA-256 fingerprint for every issued certificate and records the fingerprint with essential metadata on an Ethereum Sepolia smart contract, while the actual certificate remains stored off-chain in secure cloud storage. QR codes provide direct access to a public verification interface, where the certificate status and integrity can be checked against the institutional database and blockchain record. A Python-based document intelligence service further performs OCR, structured field extraction and consistency analysis on submitted documents to identify potential discrepancies. The platform is implemented using Flutter, Java Spring Boot, PostgreSQL, Python/FastAPI, Solidity, Ethereum, AWS, Docker and REST APIs.

The core technology map



This gives you a final-year project that is not simply "a blockchain project." It is a complete software system involving mobile development, web development, backend engineering, DBMS, cybersecurity, cryptography, cloud computing, distributed systems, blockchain, Python/AI, OCR, software testing and DevOps.

The important architectural principle: blockchain proves integrity; PostgreSQL manages application data; S3 stores documents; Spring Boot orchestrates the system; Python analyzes documents; Flutter/Web provide the user experience. That separation gives the project a technically defensible architecture rather than a collection of unrelated technologies.